funcml

8 lines · **2** framework calls

```
tuned <- tune(train, form, model = "xgboost", grid =
   TUNE_GRID,
              resampling = group_cv(v = CV_FOLDS,
                 group = folds),
              metric = if (is_classif) "auc" else
                 "rmse", seed = seed, nthread = 1)
pred <- if (is_classif) {
  predict(tuned$fit_best, newdata = test, type =
     "prob")[, "Yes"]
} else {
  predict(tuned$fit_best, newdata = test)
}
```

mlr3

26 lines · **17** framework calls

```
set.seed(seed)
tt <- if (is_classif) {
  as_task_classif(train, target = outcome, positive = "Yes")
} else {
  as_task_regr(train, target = outcome)
}
rs <- rsmp("custom_cv")
rs$instantiate(tt, f = factor(folds))
learner <- lrn(if (is_classif) "classif.xgboost" else "regr.xgboost",
   nthread = 1)
if (is_classif) learner$predict_type <- "prob"
at <- auto_tuner(
  tuner = tnr("design_points", design =
     data.table::as.data.table(TUNE_GRID)),
  learner = learner,
  resampling = rs,
  measure = msr(if (is_classif) "classif.auc" else "regr.rmse"),
  search_space = ps(nrounds = p_int(1, 1000), max_depth = p_int(1,
     20),
                    eta = p_dbl(0, 1))
)
at$train(tt)
new_tt <- if (is_classif) {
  as_task_classif(test, target = outcome, positive = "Yes")
} else {
  as_task_regr(test, target = outcome)
}
prediction <- at$predict(new_tt)
pred <- if (is_classif) prediction$prob[, "Yes"] else
   prediction$response
```

tidymodels

21 lines · **16** framework calls

```
set.seed(seed)
spec <- set_mode(
  set_engine(boost_tree(trees = tune(), tree_depth = tune(),
     learn_rate = tune()),
             "xgboost", nthread = 1),
  if (is_classif) "classification" else "regression")
wf <- workflow(recipe(stats::reformulate(".", outcome), data =
   train), spec)
grid <- data.frame(trees = TUNE_GRID$nrounds, tree_depth =
   TUNE_GRID$max_depth,
                   learn_rate = TUNE_GRID$eta)
metric <- if (is_classif) "roc_auc" else "rmse"
metrics <- if (is_classif) metric_set(roc_auc) else metric_set(rmse)
rs <- manual_rset(lapply(seq_len(CV_FOLDS), function(k) {
  make_splits(list(analysis = which(folds != k), assessment =
     which(folds == k)), train)
}), paste0("Fold", seq_len(CV_FOLDS)))
res <- tune_grid(wf, rs, grid = grid, metrics = metrics)
best <- select_best(res, metric = metric)
final <- fit(finalize_workflow(wf, best), data = train)
pred <- if (is_classif) {
  predict(final, test, type = "prob")$.pred_Yes
} else {
  predict(final, test)$.pred
}
```